Ha ancora senso insegnare a programmare nell’era dell’AI?

Ricerca di sfondo per il talk — stato dell’arte, dati, spunti

Ricerca condotta il 15 settembre 2026. Ogni numero è accompagnato dalla fonte; dove la fonte è debole o secondaria è segnalato esplicitamente.

---

0. La risposta breve (e la tesi che i dati sostengono)

Sì, ma non “la programmazione” che insegnavamo nel 2019. I dati convergono su un punto preciso: l’AI ha reso quasi gratuita la *produzione* di codice e ha reso molto più preziosa la *capacità di capirlo, valutarlo e prendersene la responsabilità*. Il problema è che la seconda capacità si costruisce solo passando attraverso la prima. Chi salta la fatica di scrivere codice non arriva alla competenza che serve per supervisionare chi lo scrive al posto suo.

Tre fatti a supporto, ciascuno da un filone diverso:

1. **Mercato del lavoro**: i posti per junior calano, ma quelli per chi sa *validare* l’AI crescono. Stanford (agosto 2026): il calo colpisce le occupazioni dove l’AI *automatizza*; dove *aumenta* il lavoratore, l’occupazione è piatta o cresce.
2. **Apprendimento**: l’RCT di Anthropic (gennaio 2026) mostra che chi impara una libreria nuova con l’AI capisce meno (50% vs 67% al quiz), *a meno che* non usi l’AI per farsi spiegare invece che per delegare.
3. **Qualità**: il 45% del codice generato dagli LLM contiene vulnerabilità (Veracode 2025–2026, nessun miglioramento tra i cicli di test). Qualcuno deve saperlo leggere.

---

1. Cosa dicono i dati su hiring e mercato del lavoro

1.1 Il “canarino nella miniera”: i giovani in occupazioni esposte all’AI

Lo studio più solido è **“Canaries in the Coal Mine?”** di Brynjolfsson, Chandar e Chen (Stanford Digital Economy Lab), basato su dati payroll ADP (25+ milioni di lavoratori USA).

- Versione agosto 2025: lavoratori 22–25 anni in occupazioni molto esposte all’AI: **–16% di occupazione relativa** (controllando shock a livello di azienda). Lavoratori esperti: stabili.
- **Sviluppatori software 22–25 anni: –20% dalla fine del 2022.** Sviluppatori più anziani: **+6–12%** nello stesso periodo.
- **Aggiornamento agosto 2026**: il gap è **salito al 19%** (dal 15% del luglio 2025). Novità importanti:

- L'aggiustamento avviene **riducendo le assunzioni di giovani, non licenziando** ("primarily through reduced hiring of young workers rather than increased separations").
- I cali sono **concentrati dove l'AI automatizza**; dove l'AI **complementa** il lavoratore l'occupazione è piatta o in crescita, soprattutto tra gli esperti.
- La **conoscenza codificata** (formale, documentata: quella dei libri e dei tutorial) è la più sostituibile; la **conoscenza tacita** (dall'esperienza) resiste.
- L'effetto persiste anche escludendo le aziende tech e le occupazioni informatiche.

Spunto per il talk: il messaggio non è "i developer spariscono", è "spariscono i primi gradini della scala". Il mestiere sta diventando *senior-only* per costruzione, e questo è un problema di pipeline che tra 5 anni sarà un problema di tutti.

Fonti: Stanford DEL, Aug 2026 update · Paper PDF Aug 2026 · Pagina pubblicazione

1.2 I neolaureati: percezione e numeri

- **Handshake, Class of 2026 (CS majors)**: **70%** almeno "somewhat pessimistic" sulla carriera (61% media di tutti i laureandi), solo **14% ottimista**. Tra i pessimisti, **64% cita la GenAI** come fattore (47% media). Eppure **42% dei laureandi CS usa GenAI ogni giorno** (24% media).
- New-grad come quota delle assunzioni Big Tech: **~7% nel 2025**, –25% vs 2023; Google e Meta assumono ~50% di neolaureati in meno rispetto al 2021 (dato da fonti secondarie di settore: SignalFire è la fonte originaria del 7%, ma va verificata prima di citarla in slide).
- Numeri che circolano ma **da trattare con cautela** (fonti: Medium, blog di bootcamp): "–67% job posting entry-level 2022–2026", "54% delle aziende ha smesso di assumere junior". Non li citerei senza risalire alla fonte primaria.

Fonti: Handshake CS spotlight · Handshake Class of 2026 outlook

1.3 Le iscrizioni a informatica calano (per la prima volta in 15+ anni)

- **National Student Clearinghouse**: iscrizioni undergraduate a Computer & Information Sciences (4 anni) **–3,6% autunno 2025**, **–8,4% primavera 2026** (calo più ripido di qualsiasi altro campo). Graduate: **–14%**.
- Il **62% dei programmi di computing** riporta iscrizioni in calo nel 2025–26 (CRA).
- Come rispondono i campus: University of Florida offre 200+ corsi AI in tutti i 16 college (71% dei laureati primavera 2026 ne ha seguito almeno uno); **Ohio State** rende obbligatoria la "AI fluency" per tutti dalla classe 2029; Colby integra l'AI trasversalmente. Il pattern: l'AI si sposta *fuori* dal dipartimento di CS, verso tutte le discipline.

Spunto: paradosso doppio. Gli studenti scappano da CS proprio mentre ogni disciplina diventa "computing + X". E il calo delle iscrizioni oggi è la carenza di senior del 2031 (Forrester prevede –20% iscrizioni CS).

Fonti: Fortune, 6 ago 2026 · Built In

1.4 Il contro-trend: chi assume junior *per via* dell'AI

- **IBM, febbraio 2026**: **triplica le assunzioni entry-level negli USA**, esplicitamente "anche per software developer e tutti quei lavori che ci dicono che l'AI può fare" (Nickle LaMoreaux, CHRO). Ma **ridisegna il ruolo**: meno coding di routine, più contatto con il cliente e **validazione degli output dell'AI**. Il senior fa "evaluation, pattern recognition, e insegna ai giovani a esercitare il giudizio". Motivazione dichiarata: chi taglia i junior oggi non avrà i mid-level tra 3–5 anni e dovrà comprarli dai concorrenti.
- Anche **Cognizant** ha annunciato aumenti di assunzioni entry-level nel 2026.
- La crescita delle assunzioni junior si sposta verso **mid-size SaaS, agenzie, aziende non-tech con team interni**, sanità, PA.

Spunto: il junior non sparisce, cambia job description. Dal "scrivi questo CRUD" al "genera 5 soluzioni, dimmi quale è giusta e perché". Questo è esattamente ciò che va insegnato.

Fonti: Axios, 13 feb 2026 · CIO.com

1.5 La domanda di sviluppatori non è crollata: è cambiata la composizione

- **Lightcast**: annunci che richiedono skill AI pagano **+28%** (~18.000 $/anno negli USA); con ≥2 skill AI **+43%**.
- Annunci con menzione AI: ~1 su 20 a inizio 2026, **45% nei ruoli data/analytics**.
- Alcune fonti di settore (Metaintro) segnalano annunci software engineer **+30% nel 2026**, ai livelli più alti da 3 anni. Fonte commerciale, da verificare, ma coerente con il quadro "la domanda si sposta, non scompare".
- Le skill più richieste: fondamentali di software engineering **più** RAG, agentic AI, MLOps, AI security, prompt engineering, data engineering.

Fonti: aiforswes Q3 2026 · Metaintro · RezScore

1.6 WEF Future of Jobs 2025 (macro)

- **39% delle competenze core cambierà entro il 2030.**
- **Analytical thinking** resta la skill n.1 (7 aziende su 10 la considerano essenziale).
- Skill in crescita più rapida: **AI & big data** (90% dei datori di lavoro prevede domanda in aumento), poi **networks & cybersecurity**, poi **technology literacy**.
- 170M posti creati / 92M eliminati → +78M netti.
- Messaggio chiave: i lavori in crescita richiedono **combinazione di skill tecniche e umane**.

Fonte: WEF Future of Jobs 2025

---

2. Dati italiani

- **Excelsior/Unioncamere 2025**: le imprese cercano 670.000 laureati e 120.000 diplomati ITS; **50,9% dei laureati e 57,3% dei diplomati ITS sono "introvabili"**. In **Ingegneria, Informatica,**

Matematica e Fisica la difficoltà di reperimento supera spesso il 55%: più di un'offerta su due rischia di restare scoperta.

- **AlmaLaurea 2026** (335.000 laureati 2025, 81 atenei): Informatica e tecnologie ICT tra i gruppi con le **più alte chance occupazionali** a 1 anno (subito dopo le professioni sanitarie all'89,4%).
- **Osservatori Polimi (AI)**: **78% delle organizzazioni fatica ad assumere**, con difficoltà principale nel trovare skill tecniche in AI, Big Data, Cybersecurity. **76% degli annunci per white-collar ad alta qualificazione richiede anche competenze AI.**
- **Randstad "Age of Augmentation" 2026** (35,8M annunci in 11 paesi 2021–2026, **427.768 in Italia**): negli annunci italiani cresce la domanda di **pensiero critico +303%**, creatività +156%, problem solving +133%, intelligenza emotiva +120%; sul fronte tecnico crescono **"AI auditing" +122%** e **quality assurance +101%**. Chi ha certificazioni/skill AI avanza di carriera fino a 2,5 volte più rapidamente.
- Quota di annunci per **junior ~4%** vs **senior ~18%** (fonte: inchiesta Today.it, estate 2026 — non ho potuto verificare la fonte primaria; usare con cautela).
- Italia **sotto la media negli AI engineer**, Milano cresce nei nuovi ruoli (fonte secondaria, luglio 2026).

Spunto: in Italia il problema non è "troppi informatici", è che le aziende non trovano quelli che cercano — e quello che cercano è cambiato: **auditing, QA, pensiero critico**. Cioè le competenze di chi *sa giudicare* il codice, non solo produrlo. Questo è un argomento fortissimo per un formatore upskill/reskill.

Fonti: Unioncamere Excelsior · Rapporto AlmaLaurea 2026 · Osservatori Polimi AI · Quotidiano.net su Randstad · Today.it inchiesta

---

3. Cosa dice la ricerca sull'apprendimento con l'AI

Questo è il cuore dell'argomento "perché insegnare ancora a programmare": l'evidenza sperimentale dice che **usare l'AI per produrre il risultato blocca l'apprendimento; usarla per capire lo accelera**. La differenza è nel *come*, e il *come* è esattamente ciò che un insegnante può progettare.

3.1 Anthropic, "How AI assistance impacts the formation of coding skills" (gennaio 2026)

RCT con **52 sviluppatori** (per lo più junior, Python settimanale da 1+ anni) che imparano una libreria nuova (Trio, async).

- Quiz di comprensione subito dopo: **AI 50% vs a mano 67%** (Cohen's d = 0,738, p = 0,01), "quasi due voti in meno".
- Velocità: il gruppo AI ~2 minuti più veloce, **non significativo**.
- Gap più grande sulle domande di **debugging**.
- **Pattern d'uso a basso punteggio (<40%)**: delega totale; reliance progressiva (inizia chiedendo, finisce delegando); debugging iterativo ("aggiusta tu").

- **Pattern ad alto punteggio (≥65%)**: genera-poi-chiedi-spiegazione; codice + spiegazione insieme; **"conceptual inquiry"** (solo domande concettuali, errori risolti da soli — secondo gruppo più veloce in assoluto).
- Conclusione degli autori: "i guadagni di produttività possono avere come costo le competenze necessarie a validare il codice scritto dall'AI"; "lo sforzo cognitivo — e persino il restare dolorosamente bloccati — è probabilmente importante per la padronanza".

Spunto: è l'azienda che vende l'agente di coding a dire che chi delega non impara. E nota il dettaglio: il pattern "conceptual inquiry" era anche tra i più veloci. Non è un trade-off velocità/apprendimento: è un problema di *abitudine d'uso*.

Fonte: Anthropic Research

3.2 Bastani et al., "Generative AI without guardrails can harm learning" (PNAS 2025)

- ~1.000 studenti di matematica delle superiori in Turchia, tre gruppi: **GPT Base** (ChatGPT normale), **GPT Tutor** (stesso modello ma con guardrail: hint invece di risposte, input degli insegnanti), controllo.
- Durante la pratica: GPT Base +48%, GPT Tutor +127% di performance.
- **All'esame senza AI: il gruppo GPT Base fa –17% rispetto a chi non aveva mai avuto l'AI.** Il gruppo Tutor: nessun danno.
- Lezione: **non "AI sì/no", ma "AI progettata come"**. Lo stesso modello, con vincoli pedagogici, elimina il danno.

Fonte: PNAS · Knowledge at Wharton

3.3 Kazemitabaar et al. (CHI 2023) — il risultato "ottimista"

- 69 novizi 10–17 anni, 45 task Python, metà con Codex. Con Codex: **1,15× completion, 1,8× punteggi**, e **nessun calo** nei task di modifica manuale; post-test a una settimana leggermente migliore (non significativo).
- Da citare per onestà: non tutta l'evidenza è negativa. Con novizi giovani e task strutturati, l'AI non ha fatto danni. La differenza con Anthropic/Bastani è probabilmente nel design del task e nella maturità degli studenti.

Fonte: arXiv 2302.07427 · Austin Henley, sintesi

3.4 Prather et al., "The Widening Gap" (ICER 2024)

- Studio qualitativo sui novizi che programmano con GenAI. Tutte le difficoltà **metacognitive** classiche (modello sbagliato del problema, falso senso di progresso) **restano, vengono amplificate, e ne emergono di nuove**.
- Gli studenti con voti e autoefficacia più alti usano l'AI per accelerare; gli altri ci si perdono. **L'AI allarga il gap tra forti e deboli.**

Spunto: per un formatore reskill è cruciale. Gli adulti in riqualificazione sono spesso proprio il gruppo "a bassa autoefficacia" che l'AI rischia di lasciare indietro.

Fonte: ACM DL · arXiv

3.5 Effetti cognitivi più generali

- **MIT Media Lab, "Your Brain on ChatGPT"** (2025, preprint, 54 partecipanti, EEG): chi scrive saggi con LLM mostra la **connettività cerebrale più debole**, minor senso di ownership, difficoltà a citare il proprio testo. Termine coniato: **"cognitive debt"**. *Campione piccolo e preprint: usare come metafora, non come prova.*
- **Microsoft Research + CMU (CHI 2025)**: survey su 319 knowledge worker, 936 casi d'uso reali. **Più fiducia nell'AI → meno pensiero critico**; più fiducia in sé stessi → più pensiero critico. Chi usa l'AI si sposta da "fare" a "verificare", ma solo se ha la competenza per verificare.
- **"The Effortless Trap"** (Brcic & Frljic, 2026): la domanda non è "permettere o vietare" ma **dove collocare** l'AI nel processo di apprendimento. Framework a 6 passi (Prime, Probe, Point, Attach, Strengthen, Test): proteggere il *primo tentativo difficile* e il *controllo finale senza aiuti*. "Effortlessness is a warning signal."

Fonti: MIT · Microsoft Research · Effortless Trap, arXiv

---

4. Lo stato dell'arte della didattica della programmazione con l'AI

4.1 La comunità di ricerca ha già dichiarato obsoleto il modello classico

- **Dagstuhl Seminar 25311 "Generative AI in Programming Education"** (2025, report pubblicato 2026; organizzatori Becker†, Craig, Denny, Kiesler, Prather): "l'approccio tradizionale di far scrivere agli studenti molti piccoli programmi corretti da un autograder è stato **probabilmente reso obsoleto** dalla GenAI".
- **Dagstuhl Seminar 27062 "What Should We Teach CS Students in the Age of AI?"** (febbraio 2027; Leinonen, Prather, Renzella, Vassar): la questione passa dalla pedagogia al **curriculum**. "I modelli focalizzati su first-principles programming e padronanza della sintassi potrebbero non preparare più adeguatamente i laureati"; le linee guida **CS2023 precedono questo shift** e non bastano. Domande aperte: quali fondamenti restano essenziali? quali skill emergenti (data-driven reasoning, infrastruttura AI)? come valutare la comprensione profonda quando "l'AI genera soluzioni perfette agli esercizi standard"?
- ITiCSE 2025 Working Group **"Beyond the Hype"** (Prather et al.): rassegna completa di ricerca, pratiche e strumenti.

Fonti: Dagstuhl 25311 · Dagstuhl 27062 · Beyond the Hype

4.2 Approcci concreti emersi (con evidenza)

a) Prompt Problems (Denny et al., SIGCSE 2024; tool "Promptly") Lo studente riceve un problema in forma *visuale* (non testuale, per evitare copia-incolla) e deve scrivere un **prompt in linguaggio naturale** tale che l'LLM generi codice che passa i test. Sviluppa: specificazione precisa, decomposizione, lettura del codice generato, debugging del prompt. Lo shift dichiarato: "dalla scrittura del codice alla **lettura, comprensione e valutazione** del codice generato dall'LLM". → arXiv 2311.05943 · Interactions with Prompt Problems

b) "Explain in Plain English" (EiPE) e "Explaining Code with a Purpose" Gli studenti spiegano cosa fa un frammento di codice; la spiegazione viene usata come prompt e la correttezza è verificata dal codice che l'LLM genera da essa. Unisce **code comprehension** e **prompting** in un solo esercizio, con autograding. → arXiv 2403.06050 · Code Generation Based Grading

c) Knowledge Markers (2026): ogni unità didattica etichettata per tipo di conoscenza — **(A) Application** (implementazione), **(S) Structure** (concetti, modelli mentali), **(P) Procedure** (metodo, decisione, verifica). Premessa: "il codice funzionante **non è più un indicatore affidabile di comprensione**". Utile per decidere dove l'AI è ammessa e dove no. → arXiv 2604.06331

d) Istruzione esplicita al prompting in CS1 (RCT, febbraio 2026): passare da "policy sull'AI" a "insegnare a usarla" con interventi scalabili. → arXiv 2602.16033

e) DURA — Demystify, Use, Reflect, Assess (experience report CS2, 2026): prima si smonta il funzionamento dell'LLM, poi lo si usa, poi si riflette, poi si valuta. → arXiv 2606.30908

f) CS50 (Harvard) e il "duck" AI: tutor AI con guardrail pedagogici (non dà soluzioni, guida), pensato per "approssimare un rapporto 1:1 docente-studente". Deployato su migliaia di studenti dal 2023. Modello di riferimento per il "GPT Tutor" di Bastani. → Teaching CS50 with AI, SIGCSE 2024

g) Code.org: da "Hour of Code" a "Hour of AI" (dicembre 2025). Partovi: l'obiettivo non è mai stato produrre software engineer, ma cittadini che capiscono come funzionano le cose; il coding resta al centro, con l'AI intrecciata. → THE Journal

h) Curricula universitari: IIT Delhi integra i code generator nel *primo* corso di programmazione (2025–26) con enfasi sull'uso responsabile; Ohio State AI fluency obbligatoria; UF 200+ corsi AI cross-disciplinari.

4.3 I temi didattici ricorrenti (sintesi)

1. **Inversione della tassonomia di Bloom**: "creare" è diventato economico; "valutare" e "analizzare" sono diventati il collo di bottiglia. Il curriculum classico va dal basso verso l'alto (sintassi → scrittura → lettura → design); quello nuovo deve allenare *lettura, critica e specifica* molto prima.
2. **Comprehension-first**: leggere codice, spiegarlo, trovare l'errore, refactorare — prima di (o insieme a) scriverlo.
3. **Specificazione come skill primaria**: il prompt è una specifica; scrivere buone specifiche richiede di saper decomporre il problema. Questo era il vero contenuto di "programmare" da sempre.
4. **Productive struggle protetta**: momenti senza AI progettati esplicitamente (primo tentativo, verifica finale), non come punizione ma come palestra.
5. **Metacognizione esplicita**: insegnare a riconoscere il "falso senso di progresso" che l'AI amplifica.
6. **AI con guardrail vs AI nuda**: la scelta dello strumento fa la differenza tra +127% e –17% (Bastani).
7. **Valutazione**: se il codice funzionante non prova più nulla, servono colloqui orali, spiegazioni, code review dal vivo, "difendi questa PR".

---

5. Produttività e qualità: perché serve ancora qualcuno che capisca

Questi dati servono a smontare la premessa “l’AI scrive il codice, quindi non serve saperlo scrivere”.

- **METR (luglio 2025)**: RCT su sviluppatori open-source esperti sui *propri* repository: con AI **+19% tempo** (IC +2%/+39%). Si aspettavano –24%; *dopo* l’esperimento credevano ancora di essere stati –20% più veloci. **Follow-up 2026**: sui partecipanti che sono tornati, stima –18% (IC –38%/+9%), ma METR stessa dice che gli effetti di selezione rendono il dato “very weak evidence” e ha cambiato il design dello studio. Lezione robusta: **la percezione di produttività è inaffidabile**. → METR 2025 · METR update feb 2026
- **DORA 2025 “State of AI-assisted Software Development”**: l’AI è un **amplificatore** — potenzia le organizzazioni sane e amplifica le disfunzioni di quelle malate. Il valore non sta nello strumento ma nelle pratiche attorno (7 capability: AI stance chiara, dati sani, dati interni accessibili all’AI, version control forte, **small batches**, focus utente, piattaforme interne di qualità). Nessuna di queste è “saper promptare”. → DORA 2025
- **Veracode 2025–2026**: 80 task × 100+ LLM: **45% del codice generato è vulnerabile**; **Java >70%**; Python/C#/JS 38–45%. XSS e log injection peggiorano. **Nessun miglioramento tra i cicli 2025 → primavera 2026** nonostante i claim dei vendor. → Veracode Spring 2026 · Report 2025
- **GitClear (ottobre 2025)**: lo sviluppatore medio committa **+75% di codice rispetto al 2022**; il guadagno di produttività (~10%) “è reale, ma lo sono anche i costi”: cresce il codice da mantenere, la duplicazione, il churn.
- **Stack Overflow Developer Survey 2025**: **84%** usa o pianifica di usare AI, ma la fiducia scende al **29%** (–11 punti); gli sviluppatori esperti sono i più scettici (**2,6%** “highly trust”, **20%** “highly distrust”). La frustrazione n.1: soluzioni “**quasi giuste, ma non del tutto**” (66%); 45% dice che debuggare codice AI richiede più tempo. Il **52%** non usa agenti o usa solo strumenti semplici. La survey 2026 (aperta giugno 2026, “for human developers only”) ha sezioni nuove su agenti e ROI: controllare se esce prima del talk. → SO 2025 · Trust gap, feb 2026 · Survey 2026 annuncio
- **GitHub Octoverse 2025**: **36M nuovi developer** in un anno (uno al secondo), 180M+ totali; **80% dei nuovi developer usa Copilot nella prima settimana**; TypeScript supera Python e JS come linguaggio n.1 (spinto dall’AI); Python +48,8% contributor, alla base di metà dei nuovi repo AI. → GitHub Blog
- **Anthropic Economic Index (2026)**: su Claude Code il **79% delle conversazioni è “automation”** (l’AI fa il lavoro) vs 21% augmentation; via API enterprise 75% automation; su Claude.ai 49–52% augmentation. Il coding è ~un terzo del traffico consumer e ~metà di quello API. Tasso di completamento auto-valutato: 61% per il software development (vs 78% per task personali). → Economic Index gen 2026 · mar 2026 “Learning curves”

Spunto: mettere in fila “80% dei nuovi dev usa l’AI dal giorno 1” + “45% del codice AI è vulnerabile” + “gli esperti sono quelli che si fidano meno” + “chi delega non impara”. La conclusione si scrive da sola: **stiamo formando una generazione che si fida di più proprio dello strumento che gli esperti temono di più, senza dargli gli strumenti per verificarlo.**

---

6. Il dibattito pubblico: voci da citare (e contrapporre)

- **Dario Amodei** (Anthropic, Axios, maggio 2025): l'AI potrebbe eliminare metà dei lavori white-collar entry-level e portare la disoccupazione al 10–20% in 1–5 anni; "entro il 2026 l'AI scriverà essenzialmente tutto il codice".
- **Jensen Huang** (Nvidia): "pretty much disagree with almost everything" Amodei dice; "everybody's jobs will be changed", ma l'AI creerà più lavori e migliori. (Nota: Huang è anche quello che nel 2024 disse che "nessuno dovrebbe più imparare a programmare".)
- **Andrew Ng**: il rischio non è che l'AI ti rubi il lavoro, è "presentarsi al lavoro come se fosse ancora il 2022"; sostiene che *tutti* dovrebbero imparare a programmare, ora più che mai.
- **Andrej Karpathy** (Sequoia Ascent, maggio 2026, "From Vibe Coding to Agentic Engineering"): **Software 3.0** = programmare gli LLM tramite prompt, contesto, tool, esempi, memoria; "programming shifts from writing code to **writing context**". Distinzione chiave: **vibe coding alza il pavimento** (tutti possono costruire software), **agentic engineering alza il soffitto** (spec, review, test, permessi per guidare agenti probabilistici). "Puoi delegare la memorizzazione delle API, ma le decisioni sulla *forma del sistema* (identità, confini di sicurezza, integrità dei dati) restano umane." E: **"understanding cannot be outsourced"**.
- **Nickle LaMoreaux** (IBM): "We are tripling our entry-level hiring, and yes, that is for software developers."

Fonti: Fortune Huang vs Amodei · Axios · Karpathy Sequoia 2026 (sue note) · Transcript

---

7. Argomenti, analogie e frame retorici utili

Questi non vengono dalle ricerche di oggi ma dalla letteratura classica; sono utili per dare profondità al discorso.

1. **Naur, "Programming as Theory Building" (1985)**: il programma non è il codice, è la *teoria* che i programmatori hanno in testa su come il codice rispecchia il mondo. Il codice senza la teoria è morto — non lo puoi modificare in modo sensato. **Il codice generato dall'AI è codice senza teoria**, a meno che qualcuno la costruisca leggendolo. Questo è forse l'argomento più forte e più elegante per il talk.
2. **Dijkstra, "On the foolishness of 'natural language programming'" (1978)**: la precisione del linguaggio formale non è un ostacolo, è lo strumento del pensiero. Si può usare in modo ironico ("Dijkstra aveva torto? o aveva ragione e stiamo per scoprirlo?") — e collegarlo a Karpathy: "writing context" è ancora scrivere specifiche precise.
3. **Le calcolatrici e l'aritmetica** (anni '70–'80): non hanno abolito l'insegnamento della matematica, ma hanno spostato l'enfasi dal calcolo al ragionamento. Analogia imperfetta ma comunicativa: la calcolatrice non sbaglia il 45% delle volte.
4. **I compilatori e l'assembly** (anni '50–'60): "i veri programmatori scrivono in assembly" → Fortran → nessuno rimpiange l'assembly, ma i migliori sanno cosa fa il compilatore. **Livello di astrazione nuovo ≠ fine della disciplina.** Però: il compilatore è deterministico e corretto; l'LLM no. È la differenza tra un'astrazione e una *delega*.

5. **Il paradosso di Jevons**: quando il costo di produrre software crolla, la domanda di software esplode (Nadella lo ha citato esplicitamente nel 2025). Più software → più software da capire, mantenere, mettere in sicurezza.
6. **“Desirable difficulties”** (Bjork): l’apprendimento richiede sforzo; ciò che rende la pratica facile la rende spesso inefficace. Collegamento diretto con l’RCT Anthropic e con “The Effortless Trap”.
7. **La scala senza i primi pioli** (“the missing rungs”): la senior-ità si costruisce facendo il lavoro da junior. Se automatizzi il lavoro da junior, da dove vengono i senior del 2031? (Charity Majors, “Generative AI is not going to build your engineering team for you”, 2024, è una buona lettura.)
8. **Conoscenza codificata vs tacita** (Stanford 2026): i tutorial, i libri, StackOverflow sono conoscenza codificata → l’AI la sostituisce. Il giudizio, il “so che questa cosa andrà storta in produzione”, è tacito → resiste. **Insegnare a programmare oggi = costruire conoscenza tacita più in fretta, non trasmettere conoscenza codificata.**
9. **Automation vs augmentation** come frame per il formatore: la stessa tecnologia, usata in due modi, produce disoccupazione o produttività. Il formatore è chi insegna il secondo modo.
10. **Il rovesciamento del “learn to code”**: dal 2012 (“Everybody should learn to code”, Obama/Code.org) al 2024 (“nobody should learn to code”, Huang) al 2026 (“Hour of AI”, IBM triplica i junior). La storia di questo pendolo, raccontata in 60 secondi, è già un’apertura.

---

8. Cos’altro può tornare utile (suggerimenti operativi)

Dati che puoi produrre tu stesso (e che nessun altro speaker avrà): - Un **micro-sondaggio ai tuoi studenti e corsisti** (upskill/reskill) prima del talk: come usano l’AI? delegano o chiedono spiegazioni? si sentono più o meno competenti? Confrontalo con i pattern dell’RCT Anthropic. Dati locali + evidenza sperimentale è una combinazione molto convincente. - Un **piccolo esperimento in aula** replicando in scala ridotta Anthropic/Bastani: due gruppi, stessa libreria nuova, quiz dopo. Anche con 20 persone, aneddoticamente potente. - Interviste a 3–5 **hiring manager italiani** (aziende che conosci): cosa chiedono ai junior nel 2026? fanno ancora live coding? chiedono di spiegare codice generato?

Cose da verificare a ridosso del talk: - **Stack Overflow Developer Survey 2026**: dovrebbe uscire tra fine estate e autunno 2026; ha sezioni nuove su agenti e ROI. Se esce prima, aggiorna i numeri su trust/adozione. - **DORA 2026**: tipicamente esce in autunno. - **Octoverse 2026**: tipicamente fine ottobre/novembre. - **METR**: nuovo design dello studio annunciato a febbraio 2026; potrebbero uscire risultati. - **Dagstuhl 27062** è a febbraio 2027, quindi dopo; ma la call/abstract è già utile come “la comunità sta ridiscutendo il curriculum a livello mondiale”.

Demo possibili (se il formato lo consente): - “Trova il bug”: mostrare codice generato da un agente che passa i test ma ha una vulnerabilità (es. XSS o log injection, dove Veracode dice che i modelli peggiorano). Chiedere al pubblico chi la vede. Poi la domanda: “chi l’avrebbe vista senza aver mai scritto codice a mano?” - Un **Prompt Problem** dal vivo con il pubblico: problema visuale, il pubblico detta il prompt, l’LLM genera, i test falliscono, si itera. Mostra che “promptare” è specificare, e specificare è programmare. - Il “**test di Naur**”: mostrare 30 righe generate dall’AI e chiedere “cosa succede se cambio questo requisito?”. Chi non ha la teoria non sa rispondere.

Contro-argomenti da anticipare (per non sembrare un insegnante che difende il proprio posto): - “Kazemitabaar ha trovato che i novizi con Codex non peggioravano” → vero; la differenza sta nel design del task e nell’età; l’evidenza su adulti e task realistici (Anthropic, Bastani) va nell’altra direction. - “METR 2026 dice che ora l’AI accelera” → METR stessa dice che è evidenza debole e ha cambiato design; e comunque velocità ≠ apprendimento. - “Il mercato junior è morto, quindi formarli è inutile” → IBM, Cognizant, Excelsior (55% introvabili in informatica), Randstad (+122% AI auditing): il junior cambia forma, non scompare; e la carenza di senior del 2031 si combatte formando junior nel 2026. - “Basta insegnare AI literacy, non programmazione” → ma l’AI literacy per chi deve *validare* codice **è** programmazione (comprensione, debugging, security). Ohio State la rende obbligatoria per tutti, il che rende il *programmatore che sa validare* ancora più distintivo.

---

9. Indice fonti primarie (per le slide)

Tema	Fonte	Dato chiave
Hiring giovani	Stanford DEL “Canaries”, ago 2026	22–25 in occupazioni esposte: –19%; sw dev –20%; via mancate assunzioni
Percezione neolaureati	Handshake Class of 2026	70% CS pessimisti, 64% per GenAI, 42% usa GenAI ogni giorno
Iscrizioni CS	National Student Clearinghouse via Fortune	–8,4% primavera 2026, grad –14%
Contro-trend	IBM/Axios feb 2026	Triplica entry-level, ruolo ridisegnato su validazione AI
Italia mismatch	Excelsior/Unioncamere 2025	50,9% laureati introvabili; >55% in informatica
Italia occupazione	AlmaLaurea 2026	Informatica tra i tassi più alti a 1 anno
Italia skill	Randstad Age of Augmentation 2026	Pensiero critico +303%, AI auditing +122%, QA +101%
Italia aziende	Osservatori Polimi	78% fatica ad assumere; 76% annunci high-skill chiede AI
Apprendimento	Anthropic RCT gen 2026	50% vs 67%; delega = <40%, inquiry = ≥65%
Apprendimento	Bastani PNAS 2025	GPT nudo –17% all’esame; con guardrail nessun danno
Apprendimento	Prather “Widening Gap” ICER 2024	L’AI allarga il gap tra forti e deboli
Cognizione	Microsoft/CMU CHI 2025	Più fiducia nell’AI → meno pensiero critico
Produttività	METR 2025/2026	–19% con percezione +20%; follow-up inconcludente
Pratiche	DORA 2025	AI è un amplificatore; 7 capability
Sicurezza	Veracode 2025–26	45% vulnerabile, Java >70%, nessun miglioramento

Tema	Fonte	Dato chiave
Volume	GitClear 2025	+75% codice per dev vs 2022
Adozione/fiducia	Stack Overflow 2025	84% adozione, 29% fiducia, esperti 2,6% “highly trust”
Nuovi dev	Octoverse 2025	36M nuovi, 80% usa Copilot in settimana 1
Uso agenti	Anthropic Economic Index 2026	Claude Code 79% automation
Didattica	Dagstuhl 25311 / 27062	Modello classico obsoleto; ridefinire curriculum
Didattica	Denny Prompt Problems, EiPE, Knowledge Markers, Effortless Trap	Comprehension-first, specifica, productive struggle
Macro	WEF Future of Jobs 2025	39% skill cambiano; analytical thinking n.1
Visione	Karpathy Sequoia 2026	Vibe coding alza il pavimento, agentic engineering il soffitto; “understanding cannot be outsourced”